

Responsibility Assignment Patterns

Student Study Edition · Golden Master design · 13
structured lessons with original course visuals, examples,
activities, and study aids.

01 · Responsibility
Assignment & Design
Patterns

02 · Singleton — One
Controlled Instance

03 · Builder —
Construct Complex
Objects Step by Step

04 · Factory Method —
Let Subclasses Choose
the Product

05 · Facade — One
Simple Door to a
Complex Subsystem

06 · Adapter — Make
Incompatible
Interfaces Collaborate

TOPIC 11 · STUDY MAP

Contents

Use this map to move through the topic in the same lesson order as The Educator course pages.

01	Responsibility Assignment & Design Patterns	3
02	Singleton — One Controlled Instance	6
03	Builder — Construct Complex Objects Step by Step	7
04	Factory Method — Let Subclasses Choose the Product	9
05	Facade — One Simple Door to a Complex Subsystem	10
06	Adapter — Make Incompatible Interfaces Collaborate	12
07	Observer — Subscribe to State Changes	14
08	Controller — Who Handles a System Event?	16
09	Controller Quality — Delegate, Don't Become Bloated	20
10	Information Expert — Put Work Where the Knowledge Is	23
11	Creator — Who Should Create the Object?	28
12	Patterns Working Together — Full Checkout Story	31
13	Review & Practice — Choose the Pattern from the Design Problem	34

TOPIC 11 · LESSON 01 · 1 OF 13 LESSONS

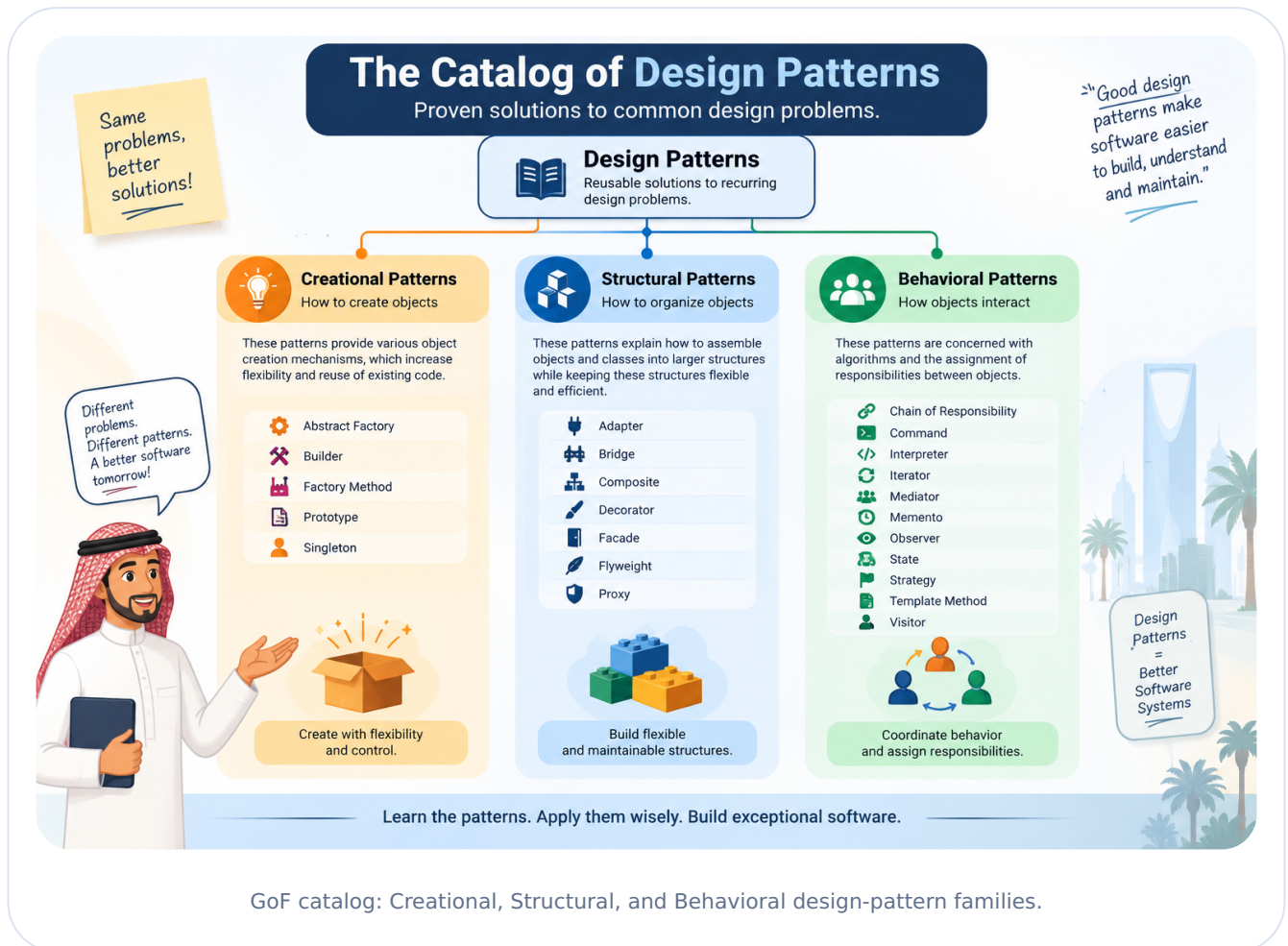
01 · PATTERN MAP

Responsibility Assignment & Design Patterns

Start with the design question, then choose the pattern family that helps you answer it.

● Core Lecture Content ● Added Study Material





DESIGN PROBLEM → PATTERN FAMILY → RESPONSIBILITY DECISION

Patterns are named design reasoning tools

This topic combines selected Gang of Four patterns with GRASP responsibility-assignment patterns.



Creational

Singleton · Builder · Factory Method



Structural

Facade · Adapter



Behavioral Observer

Pattern map

Family	Patterns in this topic	Main design question
GoF · Creational	Singleton · Builder · Factory Method	How should objects be created?
GoF · Structural	Facade · Adapter	How should structures be simplified or connected?
GoF · Behavioral	Observer	How should objects react to change?
GRASP	Controller · Information Expert · Creator	Who should receive, know, and create?

Study strategy: for every GoF pattern, remember **Intent → Problem → Solution → Structure**. For GRASP, ask **Who should own the responsibility?**

TOPIC 11 · LESSON 02 · 2 OF 13 LESSONS

02 · CREATIONAL · SINGLETON

Singleton — One Controlled Instance

Ensure that a class has only one instance while providing a global access point to it.

● Core Lecture Content ● Added Study Material



GoF · Creational

Singleton

One instance · controlled construction · shared access point

Intent

Ensure a class has only one instance and provide a global access point to it.

Problem

Multiple clients may create duplicate objects even when access to a shared resource should be controlled.

Solution

Make the default constructor private and expose a static creation/access method that creates the instance once, caches it, and returns the same object later.

Recognition cue

Private constructor + static instance field + static getInstance().

External source visual referenced by the course page

Singleton structure. Clients reach one cached instance through a public static access method.

Source: <https://refactoring.guru/design-patterns/singleton>

TOPIC 11 · LESSON 03 · 3 OF 13 LESSONS

03 · CREATIONAL · BUILDER

Builder — Construct Complex Objects Step by Step

Separate complex construction from the final object so the same process can create different representations.

● Core Lecture Content ● Added Study Material



GoF · Creational

Builder

Complex object · explicit steps · flexible representations

Intent

Construct complex objects step by step and allow different representations using the same construction code.

Problem

A complex object can force a huge constructor with many parameters and nested initialization, or scatter construction logic throughout client code.

External source visual referenced by the course page

Problem. One construction path becomes overloaded when many configurations are possible.

Source: <https://refactoring.guru/design-patterns/builder>

Solution

Extract construction code into separate Builder objects. The client or Director requests construction steps rather than relying on one monstrous constructor.

Visual hook

Director / Client → Builder steps → Product.

External source visual referenced by the course page

Solution. Construction becomes a sequence of explicit reusable steps.

Source: <https://refactoring.guru/design-patterns/builder>

TOPIC 11 · LESSON 04 · 4 OF 13 LESSONS

04 · CREATIONAL · FACTORY METHOD

Factory Method — Let Subclasses Choose the Product

Move creation behind a factory operation so subclasses can alter the concrete product without rewriting the workflow.

● Core Lecture Content ● Added Study Material



GoF · Creational

Factory Method

Also known as: Virtual Constructor

Intent

Provide an interface for creating objects in a superclass while allowing subclasses to alter the type of product created.

Problem

A logistics application originally creates `Truck` directly. Adding sea logistics introduces `Ship` and makes direct construction increasingly rigid.

Solution

Replace direct construction in the shared workflow with a factory method. Subclasses override it to return their appropriate concrete product.

External source visual referenced by the course page

Factory Method solution. The workflow depends on a product abstraction while subclasses decide which concrete product is instantiated.

Source: <https://refactoring.guru/design-patterns/factory-method>

TOPIC 11 · LESSON 05 · 5 OF 13 LESSONS

05 · STRUCTURAL · FACADE

Facade — One Simple Door to a Complex Subsystem

Provide a simplified interface to a library, framework, or other complex set of classes.

● Core Lecture Content ● Added Study Material



GoF · Structural

Facade

Simple client interface · complex subsystem behind it

External source visual referenced by the course page

Concept visual. Clients enter through a simpler interface instead of coordinating many subsystem components.

Source: <https://refactoring.guru/design-patterns/facade>

Intent

Provide a simplified interface to a complex subsystem.

Problem

Business logic may need to initialize many framework objects, track dependencies, and call operations in the correct order, creating tight coupling to third-party details.

Solution

Create a Facade that exposes only the functionality clients actually need and coordinates subsystem work internally.

Example

A video upload application may only need encode(filename, format) even if the conversion library contains dozens of classes.

External source visual referenced by the course page

Structure. The Facade coordinates the subsystem while shielding clients from unnecessary detail.

Source: <https://refactoring.guru/design-patterns/facade>

TOPIC 11 · LESSON 06 · 6 OF 13 LESSONS

06 · STRUCTURAL · ADAPTER

Adapter — Make Incompatible Interfaces Collaborate

Wrap and translate so an existing client can use an object whose interface does not match.

● Core Lecture Content ● Added Study Material



GoF · Structural

Adapter

Also known as: Wrapper

External source visual referenced by the course page

Concept. Adapter acts as a translator between incompatible interfaces.

Source: <https://refactoring.guru/design-patterns/adapter>

Intent	
Problem	A stock monitoring app supplies XML, while a new analytics library understands JSON only.

External source visual referenced by the course page

Problem. Existing client and third-party service cannot communicate directly.

Source: <https://refactoring.guru/design-patterns/adapter>

Solution

Create an Adapter that exposes the interface expected by the client, converts the request/data, and forwards it to the wrapped service.

Flow

Client → Adapter → translation → Adaptee.

External source visual referenced by the course page

Solution. Conversion is isolated in one adapter rather than spread through client code.

Source: <https://refactoring.guru/design-patterns/adapter>

Object Adapter structure

The object adapter uses composition: it implements the target interface and wraps the incompatible object.

External source visual referenced by the course page

Object Adapter. Composition lets the adapter translate while delegating real work to the wrapped service.

Source: <https://refactoring.guru/design-patterns/adapter>

TOPIC 11 · LESSON 07 · 7 OF 13 LESSONS

07 · BEHAVIORAL · OBSERVER

Observer — Subscribe to State Changes

Define a subscription mechanism so multiple objects are notified when another object changes.

● Core Lecture Content ● Added Study Material



GoF · Behavioral

Observer

Also known as: Event-Subscriber · Listener

External source visual referenced by the course page

Concept. Subscribers wait for notifications instead of repeatedly polling the publisher.

Source: <https://refactoring.guru/design-patterns/observer>

Intent

Define a subscription mechanism to notify multiple objects about events that happen to the object they observe.

Problem

A Customer wants to know when a product becomes available in a Store. Repeated polling wastes effort, but coupling Store directly to every possible customer is also undesirable.

Solution

The Publisher keeps a list of subscribers and provides subscribe/unsubscribe operations. When relevant state changes, it notifies current subscribers.

External source visual referenced by the course page

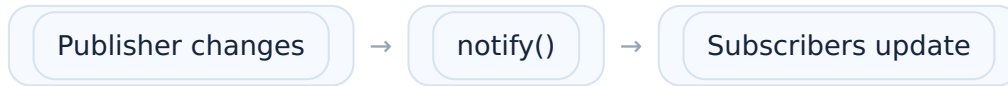
Solution. Subscribers register once and receive notifications when the Publisher changes.

Source: <https://refactoring.guru/design-patterns/observer>

External source visual referenced by the course page

Structure. Publisher depends on a subscriber interface, allowing many concrete subscribers to react independently.

Source: <https://refactoring.guru/design-patterns/observer>



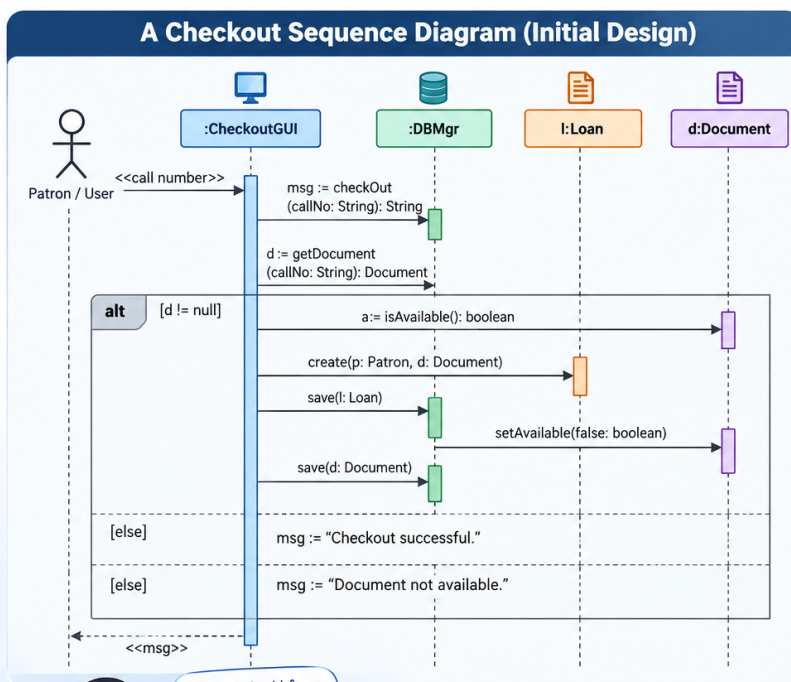
TOPIC 11 · LESSON 08 · 8 OF 13 LESSONS

08 · GRASP · CONTROLLER

Controller — Who Handles a System Event?

The chapter introduces Controller by first showing a bad checkout collaboration, then moving the system event beyond the GUI.

● Core Lecture Content ● Added Study Material



Good design assigns each class the right responsibilities.



Problems with This Design



Tight coupling between the presentation and the business objects.



The presentation has been assigned too many responsibilities.



The presentation has to handle actor requests (also called system events).



Implications

- Not designing "stupid objects."
- Changes to one may require changes to the other.
- Supporting multiple presentations is difficult and costly.



A Better Solution



Reduce or eliminate the coupling between presentation and business objects (the Low Coupling design principle).



Remove irrelevant responsibilities from the presentation (the separation of concerns principle).



Achieve high cohesion and design "stupid objects."



Have another object (class) to handle actor requests (system events).

A cleaner design is easier to maintain and easier to extend!



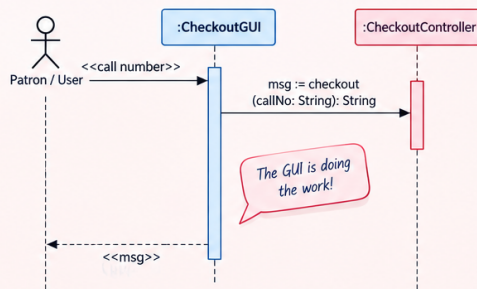
Initial checkout design showing presentation coupling and too many GUI responsibilities.

From Class Exercise to a Better Design

Use a Controller to handle the actor request and keep the UI simple.

Good Design Organizes Responsibilities!

1 Class Exercise (Initial Design) GUI handles the actor request directly



Problems with the Initial Design

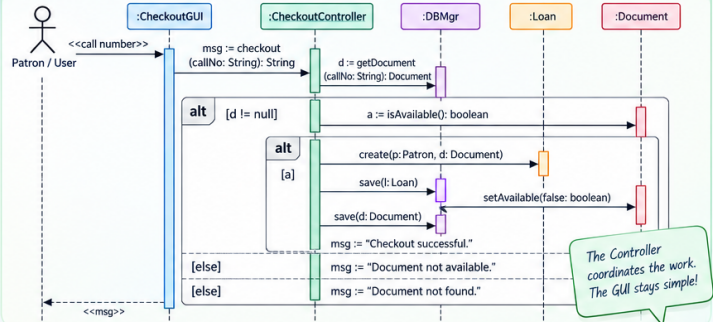
- The presentation (GUI) has to handle actor requests (system events).
- The GUI is assigned too many responsibilities.
- Tight coupling between the presentation and business objects.



The right object. The right responsibility.

A cleaner design leads to more maintainable and extensible systems.

2 The Controller Pattern (Improved Design) A Controller handles the actor request



Benefits of Using a Controller

- Assign the responsibility for handling actor requests to a Controller (e.g., :CheckoutController).
- Remove irrelevant responsibilities from the presentation (GUI).
- Apply the separation of concerns principle.
- Achieve high cohesion, low coupling, and design "stupid objects."



Controller receives the system event and coordinates the use case.

SOURCE STORY · LIBRARY CHECKOUT

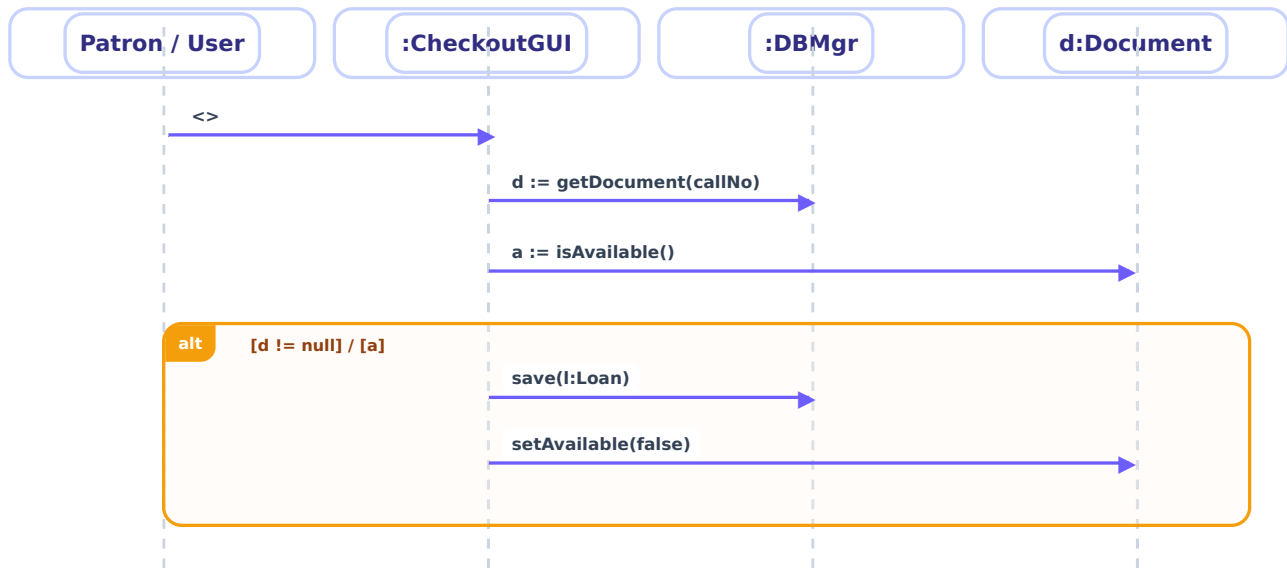
Why does CheckoutGUI need help?

The original checkout sequence has the GUI talking directly to DBMgr, Document, and Loan. The chapter labels this as tight coupling and too many presentation responsibilities.

? → WHO? → WHY?

Before Controller: presentation knows too much

SOURCE EXAMPLE · p.20



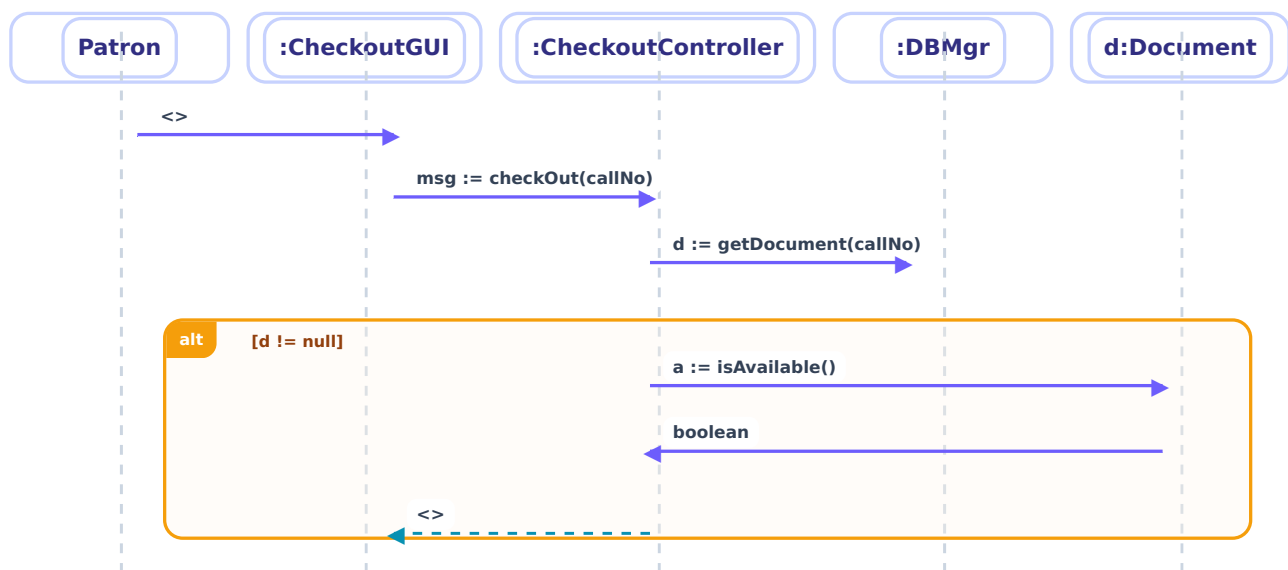
This is the design smell used by the chapter to motivate GRASP Controller: presentation is tightly coupled to business objects.

Controller pattern

Problem	Who should be responsible for handling an actor request?
Solution	Assign the responsibility to a dedicated controller.
Role	The controller receives the request from presentation, then delegates or collaborates with business objects.
Use case controller	The chapter proposes one controller per use case when appropriate; it can maintain use-case state and identify out-of-sequence events.

Controller: the system event crosses the UI boundary

SOURCE EXAMPLE · pp.23, 29-30



The complete course example on page 29 places CheckoutController between the GUI and business collaborators.

Class exercise from the chapter: complete the Checkout Document sequence diagram after introducing CheckoutController.

TOPIC 11 · LESSON 09 · 9 OF 13 LESSONS

09 · GRASP · CONTROLLER QUALITY

Controller Quality — Delegate, Don't Become Bloated

The chapter does not stop at adding a Controller. It asks what the controller should actually do and warns against procedural designs.

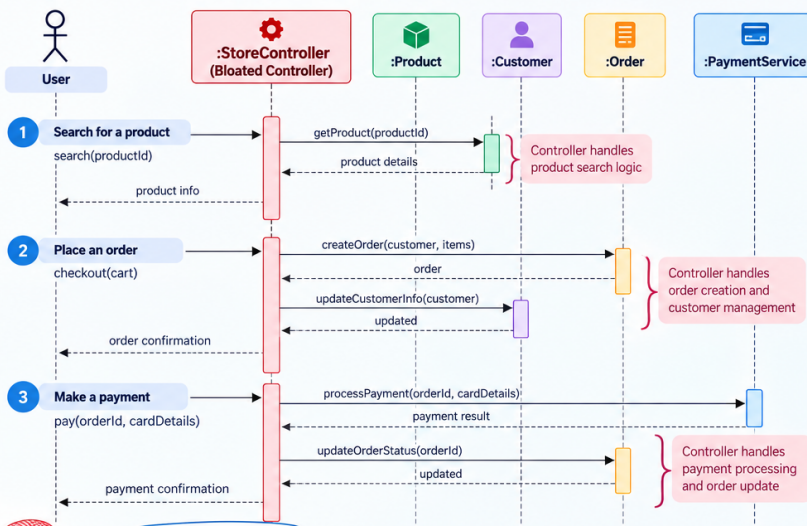
● Core Lecture Content ● Added Study Material

Bloated Controller: A Design Problem

When one controller does too much, the system becomes harder to maintain and evolve.

Sequence Diagram: Bloated Controller Example

A single controller handling multiple, unrelated actor requests.



One controller for everything? That's a red flag!



A better design delegates responsibilities to the right objects.

Keep controllers focused, simple, and maintainable.



What is a Bloated Controller?

A bloated controller is one that is assigned too many unrelated responsibilities.



Symptoms

- 1 There is only one controller to handle many actor requests.
- 2 The controller does everything to handle the actor requests rather than delegating the responsibilities to other business objects.
- 3 The controller has many attributes to store system or domain information.



Consequences

- Harder to understand and maintain.
- Low cohesion and high coupling.
- Becomes a bottleneck for changes.
- Violates the Single Responsibility Principle and separation of concerns.

"A controller should coordinate, not do everything!"

Better Design
Brighter Systems!

Bloated Controller design smell: low cohesion and too many unrelated responsibilities.

Good Design Starts with the Right Responsibility!

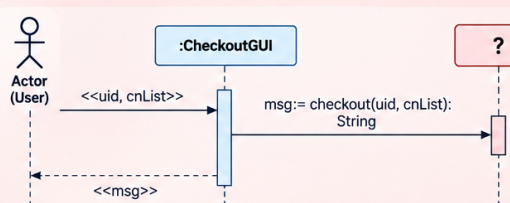
Who Should Handle an Actor Request?

It's better to use a Controller to handle system events (actor requests).

Keep the UI simple, let a controller handle the logic!

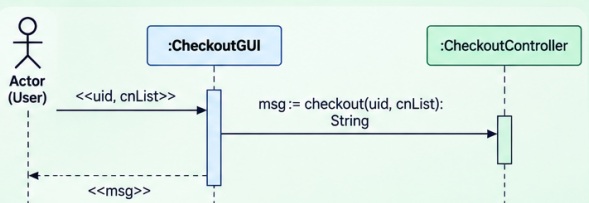
1 Initial Design

GUI handles the actor request directly



2 Better Design

A Controller handles the actor request



! Problems with the Initial Design

- The presentation (GUI) has to handle actor requests (system events).
- The GUI is assigned too many responsibilities.
- Tight coupling between the presentation and business objects.

✔ The Solution

- Assign the responsibility for handling actor requests to a Controller (e.g., :CheckoutController).
- Remove irrelevant responsibilities from the presentation (GUI).
- Apply the separation of concerns principle.
- Achieve high cohesion and design "stupid objects."
- Reduce coupling between the presentation and business objects.



Let a controller handle the system events, so the GUI can focus on the user interface — not the business logic.

Simple objects. Better systems. Brighter tomorrow!

Assign each responsibility to the right class.

A cleaner design leads to more maintainable and extensible software.

Improved checkout sequence with a focused controller and delegated responsibilities.

Healthy controller vs bloated controller

Healthy	Bloated
Coordinates one use case.	Handles many unrelated actor requests.
Delegates domain work.	Does everything itself.
Keeps use-case state when needed.	Stores substantial system/domain information.
Focused and cohesive.	Complex to understand, implement, test, and maintain.

What should it get?

What information should CheckoutController obtain from DBMgr?

Who should process?

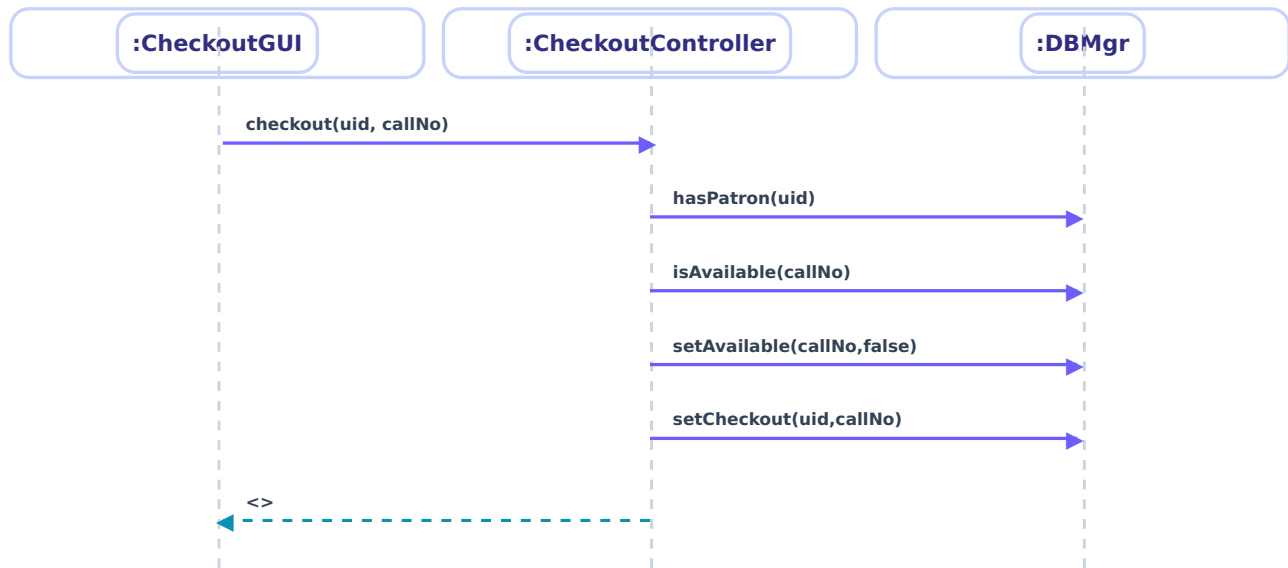
Should Controller process domain facts itself?

Who should update?

Which object actually owns the state being changed?

Conventional design: a warning example

SOURCE EXAMPLE · pp.34-36



The chapter criticizes this conventional design because responsibilities are pushed toward DBMgr in a procedural mindset rather than assigned to domain objects.

Why this matters**⚠ Procedural mindset**

Ask a data manager for facts, process them elsewhere, then send updates back.

✓ Object-oriented mindset

Find the object with the knowledge and assign the behavior there. This leads directly to Information Expert.

TOPIC 11 · LESSON 10 · 10 OF 13 LESSONS

10 · GRASP · INFORMATION EXPERT

Information Expert — Put Work Where the Knowledge Is

The chapter uses the checkout and reset-password examples to show exactly how Expert changes sequence diagrams.

● Core Lecture Content ● Added Study Material

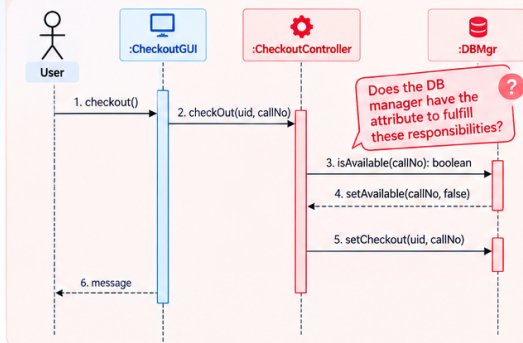
Applying The Expert Pattern

Assign responsibility to the class that has the information to fulfill it.

Ask:
"Who has the information?"

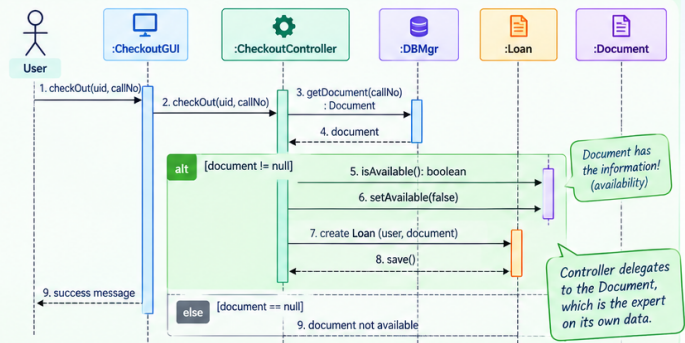
1 Initial Design (Problem)

Controller asks the DB manager for information.



2 Improved Design (Using Expert Pattern)

Controller asks the object that has the information (Document).



Problems with the Initial Design

- 1 Controller asks DB manager for availability.
- 2 DB manager may not have the attribute needed to fulfill this responsibility.
- 3 Violates the Expert Pattern.
- 4 Leads to higher coupling and lower cohesion.



What is the Expert Pattern?

Assign responsibility to the information expert — the class that has the information to fulfill the responsibility.

- ✓ Start by clearly stating the responsibility.
- ✓ Identify which class has the information.
- ✓ Assign the responsibility to that class.
- ✓ Results in higher cohesion, lower coupling, and a more maintainable design.

What Changed in the Solution?

- 1 The controller gets the Document object from DBMgr.
- 2 The controller asks Document about its availability.
- 3 The Document (the expert) handles its own data (isAvailable and setAvailable).
- 4 The controller only coordinates the process and creates a Loan.

Better Design Creates Better Systems

Let the right object do the work! That's the Expert Pattern.



"Give responsibilities to the right class — the one that knows the data."

Information Expert assigns behavior to the object that owns the needed information.



Reset Password: Apply Information Expert

Same scenario, two designs — see the problem and how Information Expert improves it.

“Assign the responsibility to the class that has the information.”

— GRASP Information Expert



Scenario

A user clicks Reset Password, enters a user id, answers an authentication question, and receives success or failure feedback.



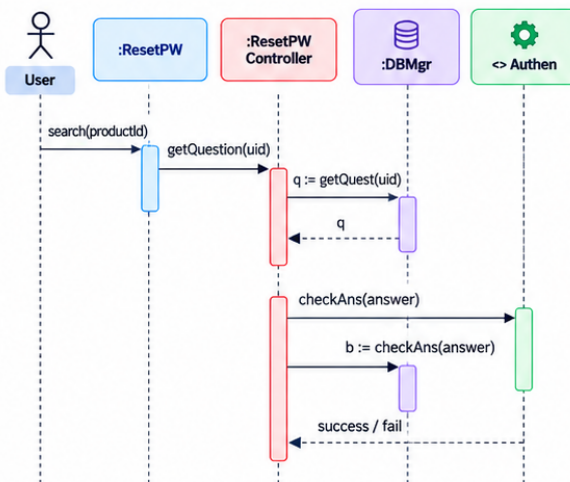
Design Goal

Delegate each responsibility to the class that has the information.



1. Problematic Design

DBMgr is asked to handle authentication logic.



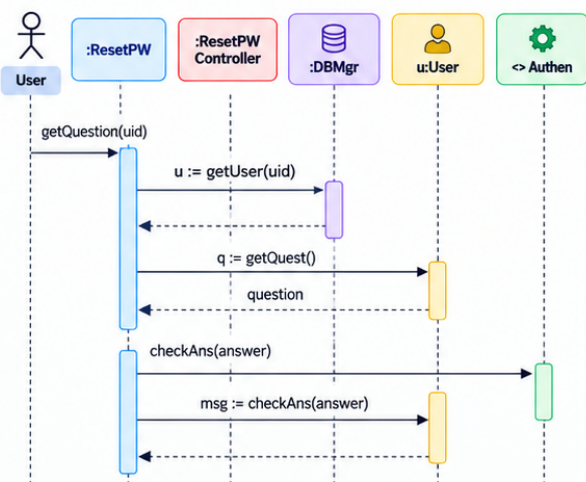
What's wrong?

- DBMgr does not own the authentication attributes.
- Controller is doing multiple, unrelated responsibilities.
- Violates the Information Expert principle.
- Leads to low cohesion and high coupling.



2. Improved Design

Delegate to User (the class that has the information).



Why this is better?

- User has the information needed to answer the question.
- Each class has a clear, focused responsibility.
- Improves cohesion and reduces coupling.
- Follows the Information Expert principle.



Key Takeaway

When designing sequence diagrams, always ask: “Which class has the information needed for this responsibility?”



Information Expert

Assign the responsibility to the class that has the information.



Reset Password example applying Information Expert.

GRASP EXPERT

Who has the information to fulfill the request?

State the responsibility first. Then assign it to the object that owns the information required to fulfill it.

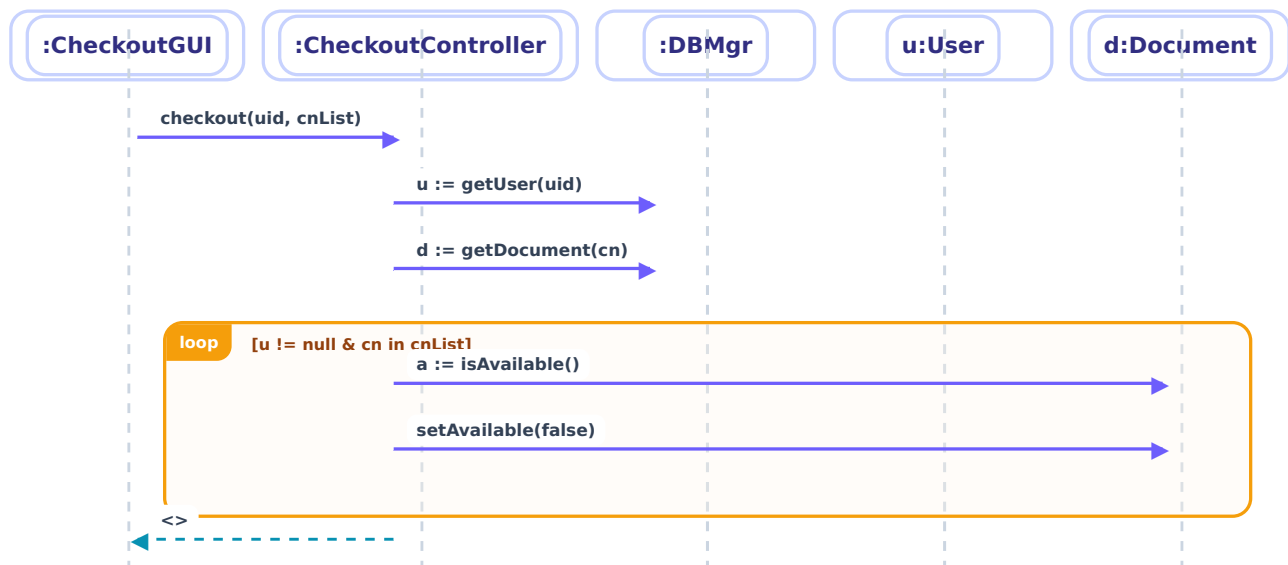
? → WHO? → WHY?

Checkout: move availability behavior to Document

Responsibility	Wrong direction	Information Expert
<code>isAvailable()</code>	DBMgr answers from raw data	Document owns availability state
<code>setAvailable(false)</code>	DBMgr updates domain state procedurally	Document changes its own state

Applying Expert to Checkout

SOURCE EXAMPLE · pp.41-42



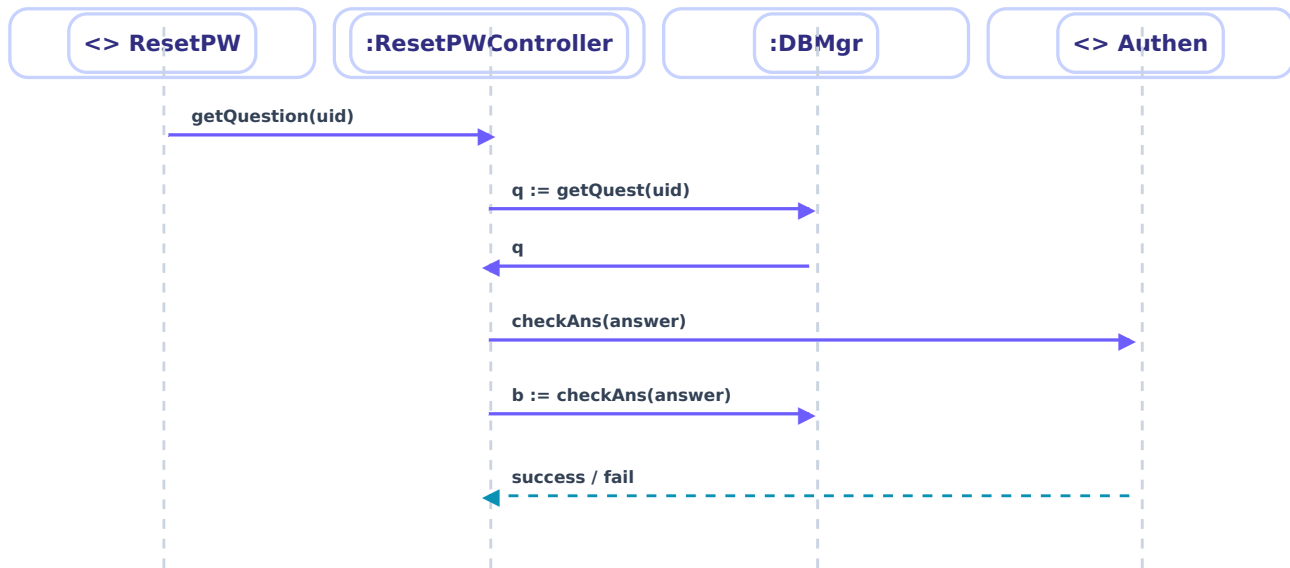
The key teaching annotation in the source is that Document has the attribute needed to answer availability; therefore it is the Information Expert.

Reset Password exercise → diagnose → redesign

Chapter exercise: User clicks Reset Password, enters user id, answers an authentication question, and receives success/failure feedback. The first proposed sequence assigns `getQuest()` and `checkAns()` to DBMgr.

Reset Password — problematic design

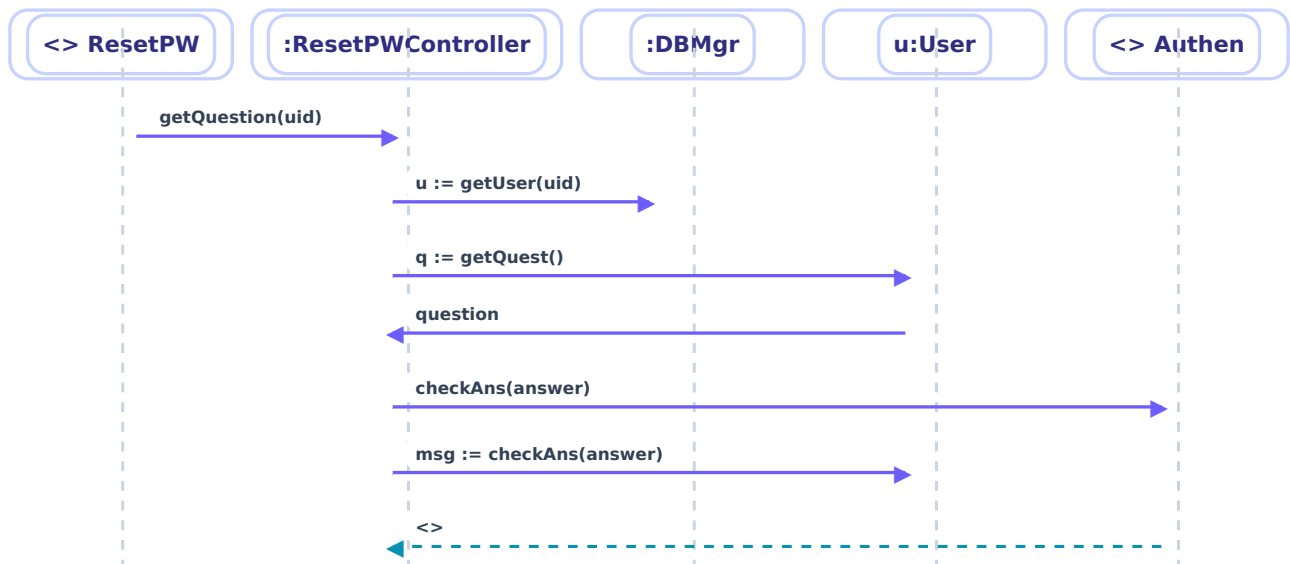
SOURCE EXAMPLE · pp.45-47



What is wrong? DBMgr does not own the authentication attributes. The chapter explicitly says this violates Expert.

Reset Password — applying Information Expert

SOURCE EXAMPLE · p.48



The corrected design delegates question/answer knowledge to User because User has the information required to fulfill those responsibilities.

Chapter takeaway: Expert is frequently applied while constructing sequence diagrams because every message forces a responsibility decision.

TOPIC 11 · LESSON 11 · 11 OF 13 LESSONS

11 · GRASP · CREATOR

Creator — Who Should Create the Object?

Object creation is a responsibility too. The chapter turns “new object” arrows into explicit design decisions.

● Core Lecture Content ● Added Study Material

The Creator Pattern

Assign the responsibility for object creation to the class that has the information to create it.

Good design knows who should create what!

Objects should be created by the class that knows the data and context!

What is the Creator Pattern?

Object creation is a common activity in object-oriented design. The Creator Pattern assigns the responsibility to create an object to the class that has the information to create it.

When to Assign the Responsibility?

Assign class B the responsibility to create an object of class A if:

- 1 B is an aggregate of A objects (e.g., a Book contains Chapters).
- 2 B contains A objects (e.g., a Dispenser contains vending items).
- 3 B records A objects (e.g., a Dispenser maintains a count for each item).
- 4 B closely uses A objects (e.g., a Game creates and uses GamePiece objects).
- 5 B has the information to create an A object (e.g., a Controller creates a Loan because it has the data).

Example: Library System — Create a Chapter

A Book creates Chapter objects because a chapter is part of a book.

Benefits

- Low coupling (the coupling already exists).
- Increased reusability.
- More maintainable and flexible design.
- A clear assignment of responsibilities.

Related Patterns

- Low Coupling (design principle)
- Other Creational Patterns: Abstract Factory, Factory Method, Builder, Prototype, Singleton
- Composite (when dealing with part-whole structures)

Key Takeaway

“Give the responsibility for creating an object to the class that knows the data and context to create it.”

A Book knows about its chapters, so it should create them!

*Better Design
Brighter Systems*

Keep your objects in the right hands!

GRASP Creator criteria and object-creation responsibility.

Creator rule

Assign class **B** responsibility to create an object of class **A** when B aggregates, contains, records, closely uses A, or has the information needed to initialize A.

◇ **Aggregates A**
B groups A objects.

◆ **Contains A**
A is a part held by B.

📁 **Records A**
B tracks A objects.

👉 **Closely uses A**
B already collaborates heavily with A.

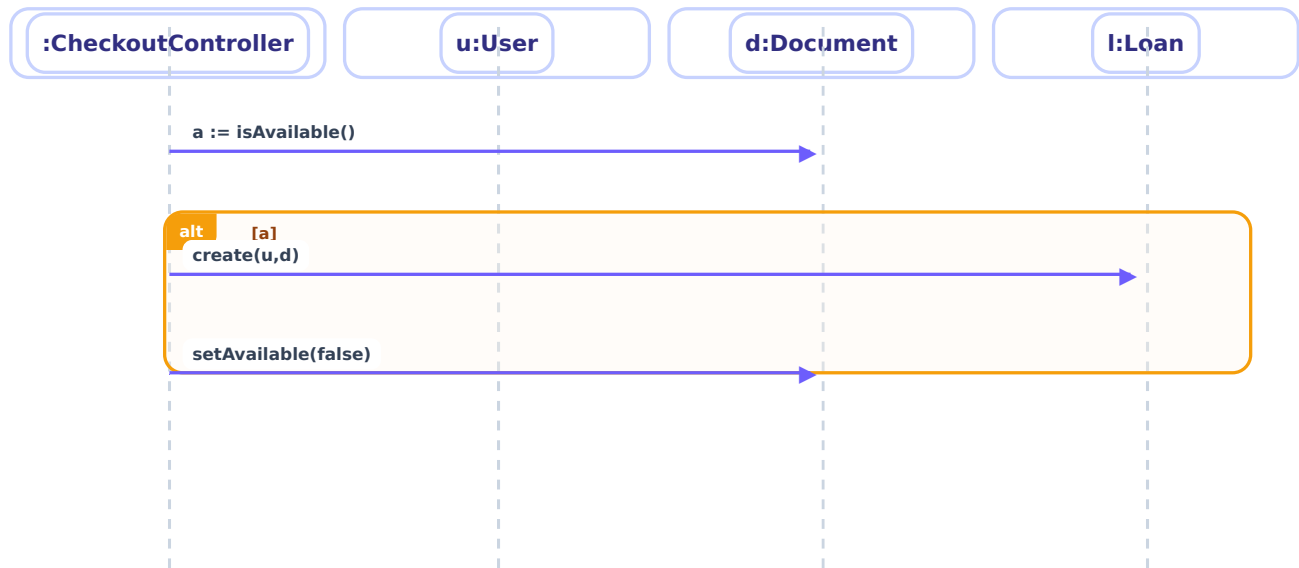
🧠 **Has creation data**
B knows A's initialization data.

The chapter's Creator activity

Object to create	Question	Course answer / reasoning
Chapter	Who should create a chapter?	Book — Chapter is a part of Book.
Loan	Who should create Loan?	CheckoutController in this example — it has the information to call Loan's constructor.
DBMgr	Who should create the database manager?	Use Creator criteria; the slide leaves this as a reasoning question.
CheckoutController	Who should create the controller?	Use the same relationship/information test rather than guessing.

Creator inside the Checkout collaboration

SOURCE EXAMPLE · pp.42, 50-52



The source explicitly notes that CheckoutController has the parameters needed to call the Loan constructor, so Creator is visible as a create message in the sequence diagram.

Benefit in the chapter: low coupling because the relationship already exists, with improved reusability.

TOPIC 11 · LESSON 12 · 12 OF 13 LESSONS

12 · INTEGRATED EXAMPLE

Patterns Working Together — Full Checkout Story

Now reconstruct the chapter's full library checkout sequence and read each message as a pattern decision.

● Core Lecture Content ● Added Study Material



Full Checkout Document Collaboration

Same scenario, two designs — see how **Information Expert** leads to a clearer design.

"Assign the responsibility to the class that has the information."
— GRASP Information Expert



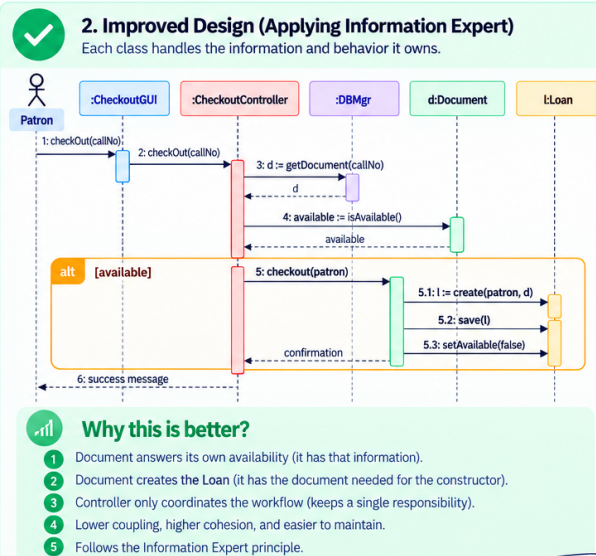
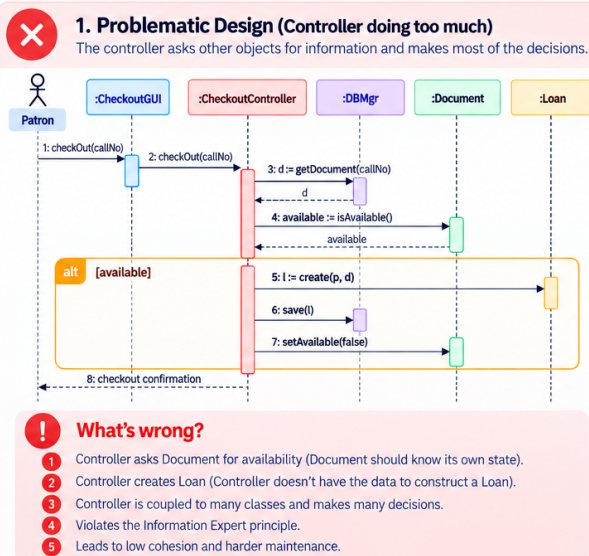
Scenario

A patron checks out a document. The system checks if the document is available. If available, it saves the checkout and creates a loan for the patron.



Design Goal

Let each class handle the information it owns.



Key Takeaway

When designing sequence diagrams, always ask:
"Which class has the information needed for this responsibility?"



Information Expert

Assign the responsibility to the class that has the information.



Integrated checkout collaboration: Controller, Information Expert, and Creator working together.

ONE USE CASE · MULTIPLE PATTERNS

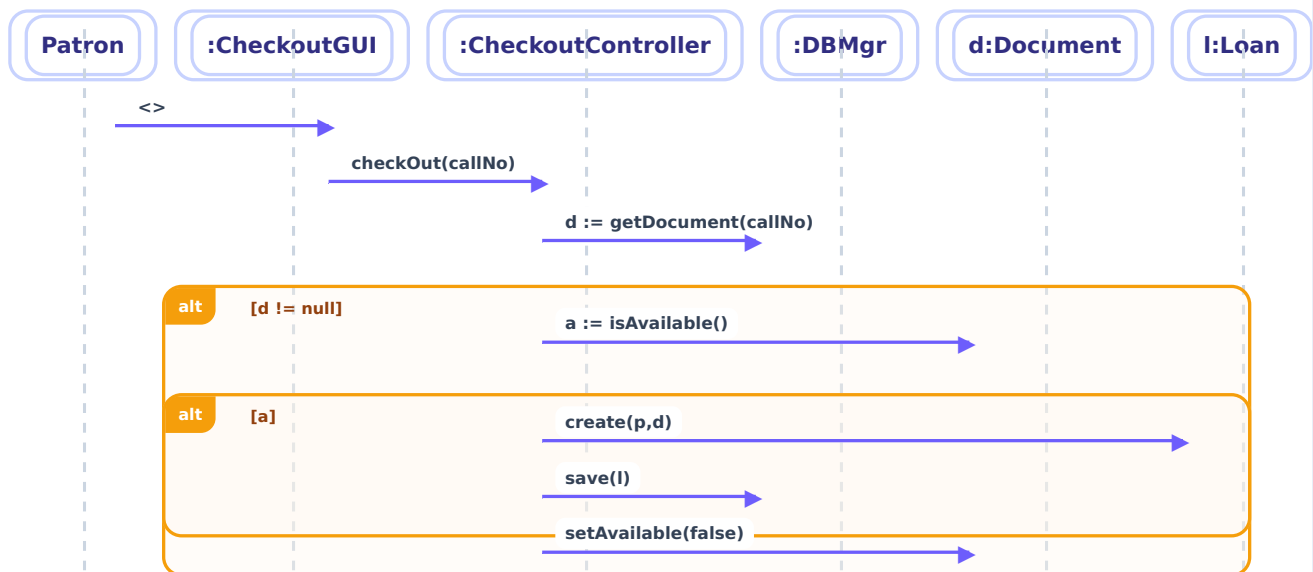
Checkout a library document

Controller decides who receives the system event. Expert decides where knowledge-based behavior belongs. Creator decides who sends the creation message.

? → WHO? → WHY?

Full Checkout Document collaboration

SOURCE EXAMPLE · p.29 + Expert refinement



Read left-to-right: UI forwards the system event to Controller; Document answers its own availability as Expert; Loan is created by the collaborator with the needed constructor information.

Annotate the sequence with patterns

Message / decision	Pattern	Why
CheckoutGUI → CheckoutController: checkOut(...)	Controller	Move actor-request handling beyond presentation.
CheckoutController → Document: isAvailable()	Expert	Document owns availability information.
CheckoutController → Loan: create(...)	Creator	Controller has the constructor information in the chapter example.

1 · Controller
Who receives?

2 · Expert
Who knows?

3 · Creator
Who creates?

4 · Quality
Low coupling? High cohesion?

Study method: cover the pattern labels, inspect each message arrow, and justify why the receiving object deserves that responsibility.

TOPIC 11 · LESSON 13 · 13 OF 13 LESSONS

13 · REVIEW

Review & Practice — Choose the Pattern from the Design Problem

Do not memorize names in isolation. Read the design problem, identify the pressure, then justify the pattern.

● Core Lecture Content ● Added Study Material

Pattern recognition table

Pattern	Family	Ask this design question
Singleton	Creational	Must there be exactly one controlled shared instance?
Builder	Creational	Is construction complex, stepwise, or highly configurable?
Factory Method	Creational	Should subclasses decide which concrete product is created?
Facade	Structural	Do clients need one simpler entry point to a complex subsystem?
Adapter	Structural	Do two useful components have incompatible interfaces?
Observer	Behavioral	Should multiple subscribers react when one publisher changes?
Controller	GRASP	Who should receive a system event from the presentation layer?
Information Expert	GRASP	Which object already has the information required to do the work?
Creator	GRASP	Which class has the strongest relationship or data needed to create this object?

Final mental model

Creational patterns control construction. **Structural** patterns organize connections. **Behavioral** patterns organize communication. **GRASP** assigns responsibilities to the right objects.

Quick design challenge

Scenario A	A third party payment SDK exposes a different interface than your application expects. → Adapter
Scenario B	A complex report can be assembled as PDF, HTML, or text through the same steps. → Builder
Scenario C	A dashboard and email service must react whenever an order changes state. → Observer
Scenario D	A checkout request arrives from the UI. Who receives the system event? → Controller
Scenario E	Which object should answer whether a Document is available? → Information Expert: Document